Efficient Intrusion Detection for In-Vehicle Networks Using Knowledge Distillation From BERT to CNN-BiLSTM

Sifan Li, Yue Cao, *Senior Member, IEEE*, Guojun Peng, Meng Li, *Senior Member, IEEE*, Wei Sun, and Luan Chen, *Member, IEEE*

***Abstract*—Under the development of intelligent transportation systems, In-Vehicle Networks (IVNs) serve as a critical channel for both internal and external communications. However, the inherent complexity and diversity of data traffic present significant challenges for the detection of IVN anomalous flows. Meanwhile, the introduction of various novel technologies has introduced new security vulnerabilities to IVNs. These vulnerabilities significantly impact the security of IVNs and the accuracy of in-vehicle Intrusion Detection Systems (IDS). To address these issues, this paper proposes a lightweight and efficient anomaly detection method based on knowledge distillation technology, termed Knowledge Distillation from BERT to CNN-BiLSTM (KDBC). Specifically, the KDBC distills the deep semantic knowledge from the BERT model into a more lightweight CNN-BiLSTM architecture, significantly reducing computational overhead and storage requirements without substantially compromising detection performance. Experimental results demonstrate that the KDBC model enhances both security and versatility, achieving superior detection accuracy in identifying abnormal attacks across diverse IVN data, including automotive Ethernet and CAN networks. Moreover, the KDBC model has been validated for its effectiveness and robustness in actual in-vehicle gateway environments, achieving an accuracy of over 0.98 and an F1 score greater than 0.98.**

***Index Terms*—Ethernet, CAN, knowledge distillation, BERT, CNN-BiLSTM.**

Received 2 December 2024; revised 4 May 2025 and 9 June 2025; accepted 10 June 2025. Date of publication 18 June 2025; date of current version 27 June 2025. This work was supported in part by Hubei Province Major Science and Technology Innovation Program under Grant 2024BAA011, in part by Horizon Europe (EU) HORIZON-TMA-Marie Skłodowska-Curie Actions (MSCA)-Staff Exchange (SE) Project TRACE-V2X under Grant 101131204, in part by Wuhan Industrial Base Innovation Program under Grant 2023010402010783, in part by the National Natural Science Foundation of China (NSFC) under Grant 62372149 and Grant U23A20303, in part by China Scholarship Council (CSC), and in part by the Key Laboratory of Knowledge Engineering with Big Data (the Ministry of Education of China) under Grant BigKEOpen2025-04. The associate editor coordinating the review of this article and approving it for publication was Prof. Guan Gui. *(Corresponding author: Yue Cao.)*

Sifan Li, Yue Cao, and Guojun Peng are with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430000, China (e-mail: sifan.li@whu.edu.cn; yue.cao@whu.edu.cn; guojpeng@whu.edu.cn).

Meng Li is with the Key Laboratory of Knowledge Engineering with Big Data, Ministry of Education, the School of Computer Science and Information Engineering, and the Intelligent Interconnected Systems Laboratory of Anhui Province, Hefei University of Technology, Hefei 230002, China, and also with HIT Center, University of Padua, 35121 Padua, Italy (e-mail: mengli@hfut.edu.cn).

Wei Sun is with the Research and Development Institute, Dongfeng Motor Corporation Ltd., Wuhan 430000, China (e-mail: sunw@dfmc.com.cn).

Luan Chen is with ETIS UMR 8051, ENSEA, CNRS, CY Cergy Paris University, 95000 Cergy, France (e-mail: luan.chen@ensea.fr).

Digital Object Identifier 10.1109/TIFS.2025.3581117

I. Introduction

With the rapid development of intelligent transportation systems, In-Vehicle Networks (IVNs) are becoming increasingly crucial in smart vehicles, enabling communication between Electronic Control Units (ECUs) and with external infrastructure like traffic lights and roadside units [1]. IVNs primarily consist of automotive Ethernet, Controller Area Network (CAN), and Time-Sensitive Networking (TSN). Automotive Ethernet, with high bandwidth and low latency, is used in systems like Advanced Driver-Assistance Systems (ADAS) and infotainment. Conversely, CAN networks remain the mainstream choice for in-vehicle control communication due to their reliability and low cost [2]. TSN meets the real-time demands of applications by providing deterministic transmission capabilities, such as autonomous driving and vehicle control systems.

Despite significant advancements in functionality and performance, IVNs face new security challenges due to their complexity and diversity [3]. The high interconnectivity of these networks makes them more susceptible to attacks. For instance, frame injection attacks in autonomous driving systems can lead to vehicles executing erroneous commands, endangering passenger safety. The broadcast nature of CAN networks makes them vulnerable to Denial-of-Service (DoS) and replay attacks, which can incapacitate critical vehicle functions [4], [5]. Additionally, time synchronization attacks in TSN can disrupt network time synchronization mechanism, leading to system failures. As new technologies continue to emerge, these security threats become more pronounced, raising the demands for robust protection of IVNs [6].

In response to the growing security threats in IVNs, various detection methods have been proposed to address these challenges. For instance, several studies have employed ensemble learning methods, such as decision trees and Support Vector Machines (SVM), to detect anomalous traffic in CAN bus networks [7]. Other research has utilized deep learning techniques, including Convolutional Neural Networks (CNN), Bidirectional Encoder Representations from Transformers (BERT) and Generative Adversarial Networks (GAN), for intrusion detection within CAN bus networks [8], [9], [10]. Furthermore, a number of studies have explored hybrid models, combining CNNs with Long Short-Term Memory (LSTM) networks to enhance detection accuracy [11], [12].

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

However, the architecture of modern IVNs has evolved significantly, incorporating a heterogeneous mix of CAN bus, automotive Ethernet, and TSN technologies to meet the growing demands for high bandwidth and low-latency communication. This multi-protocol integration introduces new and complex security challenges. Each protocol comes with distinct data formats, transmission patterns, and potential attack surfaces [13]. Consequently, IDSs that are designed to operate within a single-protocol context, such as CAN-only or Ethernet-only solutions, are no longer sufficient. They are incapable of handling the cross-domain traffic that modern centralized gateways must monitor and protect. Therefore, research efforts limited to single or dual network types fall short in addressing the comprehensive security demands of today's in-vehicle communication systems.

Although many existing IDSs perform well in isolated environments, they lack generalizability and robustness when applied to real-world scenarios involving mixed-protocol traffic. For example, a model trained exclusively on CAN traffic is unable to detect threats embedded in automotive Ethernet or TSN data streams. Meanwhile, most current approaches rely on complex model architectures, such as deep learning models like Transformer or BERT. These models inherently have high computational overhead and storage requirements, which significantly hinder their deployment in resource-constrained in-vehicle environments. In addition, many existing studies lack real-world testing, remaining theoretical or using non-representative environments. Those that test in real-world conditions often have limited IVN realization, hindering comprehensive evaluation. To fill these gaps, this paper conducts tests in a real in-vehicle gateway environment, crucial for the practical deployment of the proposed KDBC model.

Even if the BERT model demonstrates exceptional understanding and versatility capabilities for complex IVN traffic data, capturing deep semantic features, its large model size limits its direct application in resource-constrained in-vehicle environments. On the other hand, while the CNN-BiLSTM model offers a smaller model size and faster runtime, its versatility ability is weaker when handling diverse IVN attack data. Therefore, by combining BERT with CNN-BiLSTM through knowledge distillation, the superior performance of BERT can be effectively transferred to the CNN-BiLSTM model. This paper addresses the challenges of transferring the rich contextual embeddings from BERT while preserving both spatial and sequential information, and optimizes the distillation process to align BERT's complex representations with the simpler feature extraction process of CNN-BiLSTM. By carefully tuning the distillation loss function, the approach ensures minimal information loss during the transfer. As a result, the detection accuracy of CNN-BiLSTM is significantly enhanced, while maintaining its lightweight advantages, making it suitable for the real-time and resource-constrained requirements of in-vehicle systems. The main contributions of this paper are summarized as follows:

- To the best of our knowledge, the KDBC model is the first to integrate the detection of abnormal traffic across CAN networks, automotive Ethernet, and TSN into a single framework.
- Through knowledge distillation, the KDBC achieves a lightweight design while preserving the deep semantic analysis capabilities of BERT model, thereby ensuring both efficiency and detection performance.
- This study employs the TOW-IDS [6] and can-train-and-test [14] datasets for training and testing, and deploys the model on an in-vehicle gateway for practical evaluation. In both scenarios, KDBC demonstrates outstanding performance, striking a balance between lightweight design and efficiency, making it well-suited for resource-constrained in-vehicle environments.

The remainder of this paper is structured as follows: Section II outlines the current research in the field of intrusion detection for IVNs. Section III provides relevant background information on IVNs and threat models. Section IV presents a detailed description of the proposed KDBC framework and its components. Section V demonstrates the experimental evaluation of KDBC. Finally, section VI summarizes and concludes the paper.

II. Related Work

This section reviews related work, categorized into three parts based on the intrusion detection scenarios: intrusion detection research for CAN bus networks, automotive Ethernet and TSN.

A. Intrusion Detection Research for Can Bus Networks

Intrusion detection research for in-vehicle CAN bus networks is currently one of the most prominent areas in IVN studies, attracting increasing attention from industry professionals in recent years. Machine learning-based models for intrusion detection have become a mainstream approach, with a number of studies combining CNN and LSTM models for detection [11], [12]. Other research has leveraged the advantages of the BERT model from Natural Language Processing (NLP) for extracting deep semantic features in intrusion detection [15], [16]. Additionally, Du et al. [17] introduced CLUSTER, a novel approach for CAN bus intrusion detection. It utilizes the distance from known class inputs to cluster centroids as the training loss, aligning with the threshold for open set recognition. Beyond the aforementioned open set recognition techniques, transforming CAN bus data into directed graphs is also a viable option. Islam et al. [18] proposed a novel graph-based Gaussian Naive Bayes (GGNB) intrusion detection algorithm, which utilizes graph properties and PageRank-related features to achieve high detection accuracy. Furthermore, Zhang et al. [1] developed a CAN bus anomaly detection system based on Graph Neural Networks (GNN). It can detect various attacks, including message injection, suspension, and falsification, in just 3 milliseconds.

B. Intrusion Detection Research for Automotive Ethernet

Compared to intrusion detection research for in-vehicle CAN bus networks, the study of intrusion detection in automotive Ethernet has garnered relatively less attention. However, in recent years, the importance of automotive Ethernet have increasingly become evident due to the introduction of

TABLE I
COMPARISON OF RELATED WORK

Related Work	Type of IDS	Datasets	Method	Scenarios	Real Scenario Test
[11]	CNN-LSTM-based IDS	CAN Signal Extraction and Translation Dataset	CNN-LSTM with attention	CAN bus	Real vehicle test
[12]	Hybrid deep learning-based IDS	Car Hacking Dataset (CHD)	CNN, LSTM	CAN bus	No
[15]	Bert-based IDS	CHD, can-train-and-test	BERT	CAN bus	Real vehicle test
[16]	Bert-based IDS	CHD	BERT	CAN bus	No
[17]	Open set recognition-based IDS	CHD	Cluster	CAN bus	No
[18]	Gaussian naive bayes-based IDS	OTIDS	GNN	CAN bus	FPGA test
[1]	Federated GNN-based IDS	CAN Signal Extraction and Translation Dataset	GNN	CAN bus	No
[6]	Deep CNN-based IDS	TOW-IDS	CNN Wavelet Transform	Ethernet TSN	Real vehicle test
[19]	Deep learning-based IDS	TOW-IDS, AEID	RF, CNN	Ethernet TSN	No
[20]	Unsupervised-IDS	TOW-IDS	Autoencoder	Ethernet TSN	Nvidia Jetson AGX Xavier test
[21]	Open source-based IDS	Private dataset	Rule-based	TSN	TSN testbed
[22]	Anomaly-based IDS	Private dataset	Ingress control	TSN	Simulated
Proposed KDBC	Knowledge distillation-based IDS	TOW-IDS can-train-and-test private dataset	BERT CNN-BiLSTM	CAN bus Ethernet TSN	Real in-vehicle gateway test

Advanced Driver-Assistance Systems (ADAS). Han et al. [6] proposed a method for detecting abnormalities in automotive Ethernet using wavelet transform and deep CNNs, effectively addressing inherent attack vectors and vulnerabilities. Based on the dataset introduced in [6], two studies [19], [20] were conducted. First, da Luz et al. [19] proposed a novel multi-stage deep learning-based Intrusion Detection System (IDS) for automotive Ethernet networks, which combines a Random Forest (RF) with a pruned CNN for classifying attack types. Second, Jeong et al. [20] introduced the Automotive Ethernet Real-time Observer (AERO), an unsupervised network IDS designed to protect IVNs by analyzing automotive Ethernet traffic and detecting anomalies.

C. Intrusion Detection Research for TSN

Given that in-vehicle TSN primarily employs the IEEE 802.1 series of standards, this section highlights relevant intrusion detection research. Ergenç et al. [21] presented TSNZeek, the first open-source security monitoring and intrusion detection mechanism for IEEE 802.1 TSN protocols, addressing the lack of inherent security measures within these standards. Meyer et al. [22] introduced a framework for detecting anomalous traffic through ingress control in TSN, utilizing stringent TSN traffic descriptors to reduce false positives. They implemented Software-Defined Networking (SDN) technologies to gather and assess ingress anomaly reports, thereby identifying and blocking generating flows from the network.

D. Motivation

Table I summarizes and compares related studies. Most focus on either the CAN bus or TSN, with a few addressing both automotive Ethernet and TSN. However, the IVN architecture in modern vehicles varies, integrating CAN bus, automotive Ethernet, and TSN to achieve higher data rates and lower latency. Thus, research limited to single or dual IVN scenarios fails to address the full security needs of modern IVNs.

Additionally, many existing studies lack real-world testing, remaining theoretical or using non-representative environments. Those that test in real-world conditions often have limited IVN realization, hindering comprehensive evaluation. To fill these gaps, this paper conducts tests in a real in-vehicle gateway environment, crucial for the practical deployment of the proposed KDBC model.

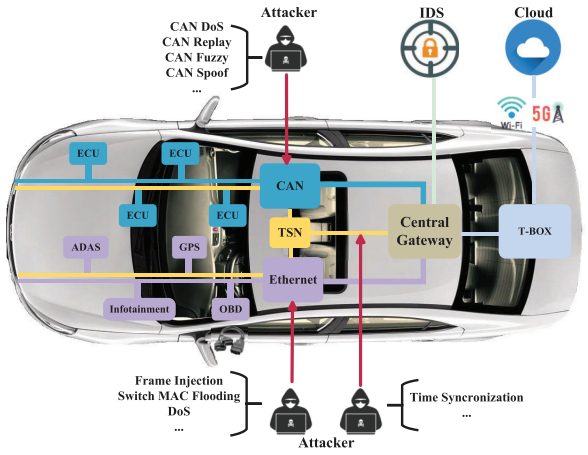


Fig. 1. The scenario of IVN.

III. SYSTEM MODEL

A. Background

As illustrated in Fig. 1, the in-vehicle central gateway, where the IDS model is deployed, primarily connects the CAN bus, automotive Ethernet, TSN, and Telematics BOX (T-BOX). The T-BOX is mainly used for collecting vehicle data, which is then transmitted to the cloud via networks such as Wi-Fi or 5G [23]. Next, this section primarily introduces three typical IVNs: automotive Ethernet, CAN bus networks, and TSN. TSN is considered an extension of automotive Ethernet, and it is discussed separately due to its susceptibility to attacks, such as time synchronization attacks.

1) Automotive Ethernet: Automotive Ethernet is a high-bandwidth, low-latency network technology developed to meet the communication needs of modern vehicles, making it ideal

for ADAS, infotainment, and large-scale data transmission [24]. Using single-pair UTP cabling, it reduces space, weight, and costs, while supporting real-time data transfer from high-resolution cameras and sensors for autonomous driving. Its adoption simplifies IVN architecture, reduces system complexity and wiring costs, and is becoming the standard for efficient in-vehicle communication in future intelligent vehicles [25].

2) Controller Area Network (CAN): CAN, developed by Bosch in the late 1980s, is a widely used protocol in IVNs, valued for its reliability, real-time performance, and low cost [26]. It uses a multi-master, message-oriented protocol to facilitate communication between ECUs, with a simple frame structure and robust error detection to ensure data accuracy and stability [27]. Despite bandwidth limitations as vehicle systems grow more complex, enhanced protocols like CAN-FD maintain CAN's vital role in modern vehicles, ensuring safety and reliability [28].

3) Time-Sensitive Networking (TSN): TSN is a set of Ethernet-based standards developed under IEEE 802.1 to provide deterministic data transmission and time synchronization, ideal for applications requiring low latency and jitter [29]. It uses sub-standards like Time-Aware Shaping, Frame Preemption, and Precision Time Protocol to manage network traffic, ensuring reliable communication for critical applications in IVNs, such as autonomous driving and ADAS. By reducing latency and packet loss, TSN enhances vehicle safety and performance, supporting the development of intelligent, connected vehicles in complex environments.

B. Threat Model

As shown in Fig. 1, attacks on automotive Ethernet are primarily categorized into frame injection attacks, switch MAC flooding attacks, and DoS attacks. In contrast, attacks on the CAN bus are mainly classified as DoS attacks, replay attacks, fuzzy attacks, gear spoof attacks, and RPM spoof attacks. Attacks targeting TSN predominantly involve time synchronization attacks.

1) Frame Injection Attack: An adversary sends forged data frames to disrupt communication, potentially altering or losing critical control information and causing incorrect vehicle system responses. This attack targets the integrity of data in transit, possibly causing discrepancies in-vehicle control functions, which may lead to unpredictable behavior or system failures. It is particularly dangerous in real-time systems, where timing and accuracy of communication are paramount for safety.

2) Switch MAC Flooding Attack: An adversary overwhelms a switch's forwarding table with forged MAC addresses, preventing it from properly forwarding legitimate packets. The goal of this attack is to disrupt the network's ability to forward data correctly, leading to network congestion, delayed packet delivery, or a complete failure to transmit data. This attack can degrade network performance, causing disruptions in vehicle communications, such as sensor or control message delays, which can compromise vehicle operation.

3) DoS Attack: DoS attacks can target either automotive Ethernet or the CAN bus, with Ethernet attacks involving the transmission of invalid data frames that consume bandwidth and disrupt legitimate communication, affecting vehicle safety. In CAN bus attacks, sending numerous high-priority messages blocks the bus, delaying critical control commands and jeopardizing driving safety [30]. This attack can lead to system failures or performance degradation, resulting in critical control commands being delayed and jeopardizing driving safety.

4) Replay Attack: Attackers resend intercepted valid data packets to deceive the system into repeating operations, causing state confusion or unpredictable behavior. The attack can confuse the state machine of vehicle, leading to erroneous actions, such as repeating an earlier maneuver or applying an outdated control input.

5) Fuzzy Attack: A fuzzy attack exploits uncertainties in sensor or control system data by injecting random or ambiguous inputs, causing confusion in decision-making and potentially compromising vehicle security. This could degrade vehicle safety by misinterpreting sensor data, leading to incorrect navigation decisions or unsafe vehicle operations.

6) Gear Spoof Attack: Attackers manipulate sensor or ECU data to falsely report the vehicle's gear position, misleading the driver or vehicle system into misinterpreting the current gear status. This attack can cause incorrect gear shifting, leading to unsafe vehicle operation, such as unintended acceleration, stalling, or even vehicle collisions due to inaccurate speed or power delivery information. It can severely compromise driver safety, particularly in automated or semi-automated systems where gear shifts are handled autonomously.

7) RPM Spoof Attack: Attackers can manipulate sensor or ECU data to transmit false tachometer readings, misleading engine speed information. This attack can distort the driver's perception of engine performance, potentially leading to unsafe driving decisions, such as over-revving the engine or failing to recognize an engine failure. By manipulating engine speed readings, the attacker can disrupt the operational efficiency of vehicle, increasing the risk of accidents or mechanical damage.

8) Time Synchronization Attack: The attacker disrupts time synchronization mechanisms, such as the Precision Time Protocol (PTP), causing discrepancies in time-stamped data used by autonomous or driver-assist systems. Time synchronization is critical for systems that rely on time-sensitive data, such as collision avoidance, navigation, or autonomous driving. Disrupting time synchronization can lead to misalignment of sensor data, causing delays in decision-making or faulty system responses, which can directly endanger vehicle safety by impairing the accuracy and timing of control actions.

To identify threat models across multiple IVNs, the KDBC model needs to have strong versatility, capable of recognizing different attacks in various IVN environments. To achieve this, the KDBC model adopts a binary classification design, where the output is classified as either normal or abnormal based on traffic analysis. Additionally, during the initial training phase, the KDBC model is trained using various publicly available IVN datasets. Once the model achieves stable performance, it is then deployed in a real-world in-vehicle gateway environment for practical testing.

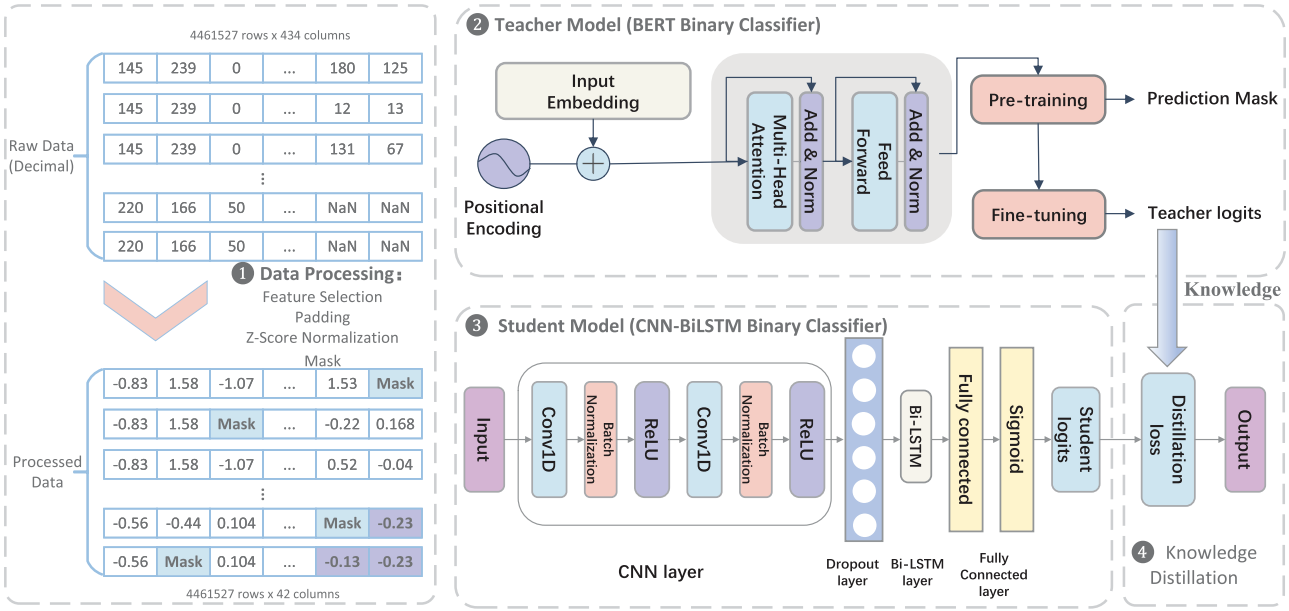


Fig. 2. The framework of KDBC.

IV. Proposed KDBC Framework

A. Overview

The proposed KDBC primarily aims to address the intrusion detection challenges associated with diverse network data in IVNs. Therefore, its optimal deployment location is at the in-vehicle central gateway. As illustrated in Fig. 2, the KDBC framework consists of three main components: data processing, the teacher model (BERT-based binary classifier), and the student model (CNN-BiLSTM-based binary classifier), along with the knowledge distillation process.

Initially, the KDBC reads traffic files (.pcap) collected from the in-vehicle gateway and converts them into decimal text data. Subsequently, data preprocessing is conducted, which includes feature selection, padding, and masking. The processed data is then simultaneously fed into both the teacher and student models for training and testing. Finally, the KDBC computes the distillation loss based on the logits from the teacher and student models, and outputs the results once the distillation loss meets the desired threshold.

In the proposed KDBC architecture, each component plays a distinct yet collaborative role to enable efficient and accurate intrusion detection in IVNs. The BERT-based teacher model serves as a powerful semantic feature extractor, leveraging its deep bidirectional transformer layers to capture complex contextual relationships in the input data. Meanwhile, the student model, composed of a CNN for local feature extraction and a BiLSTM for capturing temporal dependencies, provides a lightweight and deployable solution suitable for resource-constrained environments like in-vehicle gateways. During training, both models receive the same preprocessed input data. The knowledge distillation mechanism bridges these models by transferring the knowledge from the teacher to the student: it computes the distillation loss between the soft output logits of the teacher and student models, allowing the student to learn not only from the ground truth but also from the richer representations learned by the teacher. This architecture ensures that the student model maintains high detection performance while significantly reducing computational and memory costs, making it ideal for real-world IVN deployment.

B. Data Preprocessing

Data preprocessing is a critical step to ensure model performance and detection accuracy. Given the complex and variable nature of IVN environments, the collected data often suffer from issues such as missing values, varying lengths, and feature redundancy. Consequently, a series of preprocessing techniques are necessary to optimize data quality. This section will provide a detailed overview of four core preprocessing strategies: feature selection, Z-Score Normalization, zero padding, and masked language data augmentation.

1) Feature Selection: Feature selection is an indispensable component of data preprocessing, aiming to identify the most representative subset of features from the raw data. It can also mitigate the impact of redundant information on model performance. In IVN data, the number of features is often large and complex, encompassing many that are irrelevant or weakly related to intrusion detection. In addition, feature selection can effectively reduce the complexity of subsequent tasks such as padding and masking, making it easier to unify data length through padding.

The KDBC framework utilizes mutual information-based feature selection. Specifically, mutual information quantifies the degree to which knowing one variable reduces the uncertainty of another variable [31]. By calculating the mutual information between features and the target variable (traffic labels), the KDBC can assess the contribution of each feature to the target variable, thereby selecting the most relevant features. For each variable A and the target variable B, the mutual information $I(A;B)$ is defined as:

$$I(A;B) = \sum_{a\in A}\sum_{b\in B} d(a,b)\log\left(\frac{d(a,b)}{d(a)d(b)}\right) \tag{1}$$

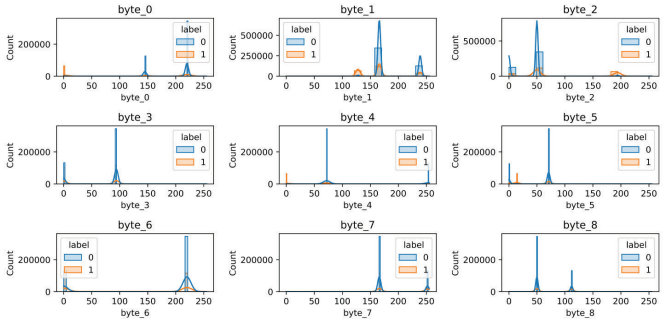


Fig. 3. Feature distribution by label.

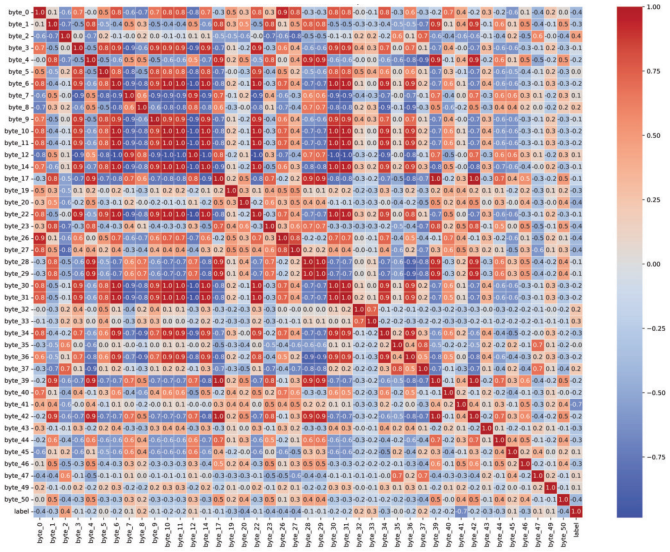


Fig. 4. Feature correlation heatmap.

Here, $d(a, b)$ represents the joint probability distribution of A and B, while $d(a)$ and $d(b)$ denote the marginal probability distributions of A and B, respectively. The mutual information between each feature and the target variable in the dataset is computed using Equation 1. The features are ranked in descending order according to their mutual information values with the target variable to assess their relative importance. Finally, 42 features with mutual information values greater than 0.1 are selected for inclusion based on this ranking.

After feature selection, Fig. 3 illustrates the distribution of different label data values for the top 9 selected features. For instance, regarding the first feature, it is observed that a portion of the malicious traffic falls within the range of 0. This is attributed to the presence of certain DoS attacks on the CAN bus within the malicious traffic. Fig. 4 displays the heatmap of the 42 selected features, with values ranging from $[-1, 1]$. Values closer to -1 or 1 indicate a strong negative or positive correlation between two features, while values near 0 suggest a weak correlation.

2) Zero Padding: Zero padding is primarily employed to address missing values and achieve uniformity in data length [32]. In IVN data, issues such as device failures, communication interruptions, or inconsistent sampling frequencies often result in missing data (value is NULL). The zero-padding operation fills these gaps with zero values, effectively restoring data integrity while ensuring that all samples have the same length for subsequent processing, which facilitates batch processing and model training. Additionally, zero padding can mitigate noise interference arising from length discrepancies, thereby enhancing the versatility capability of model.

After the feature selection process, the data length from both automotive Ethernet and TSN networks was unified to 42 bytes. Therefore, the data length from the CAN bus needs to be extended to 42 bytes using zero padding, while retaining the original 16-byte content. Zero padding is instrumental in enabling intrusion detection models to process data from multiple IVN protocols by standardizing input length, thereby ensuring compatibility across heterogeneous data formats such as CAN, Ethernet, and TSN. This alignment facilitates the ability of model to recognize cross-protocol anomalies without structural inconsistencies, enhancing the versatility and robustness of the KDBC across diverse network traffic.

3) Z-Score Normalization: Z-score normalization, also known as standard deviation normalization, is a method that transforms data into a standard normal distribution through mathematical conversion [33]. Its primary purpose is to convert data of varying magnitudes into a unified scale of Z-score values, enhancing data comparability and reducing interpretability bias. In this study, to mitigate the impact of varying data ranges across different protocols, this study applies Z-score normalization to the data, as defined by the following formula:

$$Score = \frac{S - \mu}{SD} \tag{2}$$

where $Score$ represents the standardized Z-score, a dimensionless value. S represents an individual observation, i.e., a data point in the original dataset. μ denotes the mean of the entire dataset. SD represents the standard deviation of the dataset.

4) Masked Language Data Augmentation: The masking data augmentation technique is an innovative method inspired by NLP, aimed at enhancing the robustness and versatility capability of model by randomly masking certain data features. In the context of IVN data, the KDBC can consider continuous or discrete features (such as network traffic) as "vocabulary", and simulate real-world noise/anomalies by randomly selecting and masking these "vocabularies". This approach compels the model to learn and make predictions with partial information, thereby stimulating its ability to explore additional latent features and correlations.

Masking data augmentation not only improves the capacity of model to recognize unknown attacks but also mitigates overfitting to extent, enhancing the versatility performance of model. As shown in Algorithm 1, in the KDBC framework, the mask probability is set to 10%. An empty list, *masked_data*, is initialized, and 10% of the traffic is randomly sampled from the overall data (lines 1-5). Among the selected traffic, 70% will randomly replace one token with [mask] (lines 7-9), 20% will randomly replace one token with another random token (lines 10-11), and 10% will remain unchanged (lines 12-13).

C. Teacher Model

The choice of BERT model as the teacher model among numerous deep learning models is based on several key

Algorithm 1 Masked Language Data Augmentation Algorithm

Input: *text_corpus* (a collection of text documents)
Output: *modified_corpus* (text documents with tokens masked or replaced)

Initialize *modified_corpus* as an empty list
for each text in *text_corpus* **do**
  Generate a random number r between 0 and 1
  if $r < 0.1$ **then**
    Append text to *modified_corpus*
    Generate another random number p between 0 and 1
    if $p < 0.7$ **then**
      Randomly select a token from the text
      Replace the selected token with [mask]
    else if $p < 0.9$ **then**
      Randomly select a token from the text
      Replace the selected token with another random token from the vocabulary
    else
      Keep the text unchanged
    end if
  else
    Append the unchanged text to *modified_corpus*
  end if
end for
return: *modified_corpus*

reasons. First, unlike traditional feature engineering-based intrusion detection methods, BERT can automatically extract useful features from network traffic data without the need for complex feature extraction rules [34]. It can significantly enhance the efficiency and accuracy of intrusion detection. Second, the pre-training of BERT on a large corpus endows it with strong cross-domain adaptability. In the context of IVN traffic intrusion detection, BERT can quickly adapt to and handle various types of network traffic data, including encrypted traffic [35]. Finally, the BERT model supports fine-tuning, allowing for further training and optimization tailored to specific tasks. In the realm of IVN traffic intrusion detection, fine-tuning the BERT model can improve detection accuracy and efficiency by adapting to different network environments.

The BERT model is a pre-trained deep neural network that centers around the bidirectional encoder of the Transformer architecture. This model generates high-quality language representations through unsupervised pre-training on large-scale corpora, followed by fine-tuning for specific tasks.

1) Input Embedding: The input embedding of BERT model for IVN traffic consists of three processes: token embedding, segment embedding, and position embedding. The input traffic is initially segmented into tokens using a tokenizer, with each token mapped to a high-dimensional space to create token embedding. Segment embedding adds an additional embedding for each token to indicate its association with a specific traffic stream. Position embedding provides a unique embedding vector for each position, which are learned during the training process.

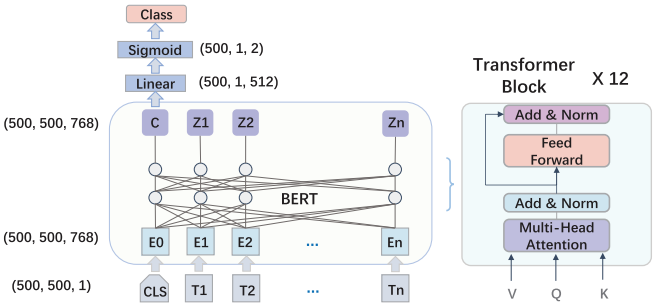


Fig. 5. The structure of BERT model.

2) Transformer Encoders: As shown in Fig. 5, the core of BERT model is the encoder part of the Transformer, which consists of multiple Transformer layers. Each Transformer layer primarily comprises a multi-head self-attention mechanism and a Feedforward Neural Network (FFNN).

Multi-Head Self-Attention Mechanism: The self-attention mechanism allows the model to consider all other features in the input sequence while processing each individual feature. The multi-head self-attention mechanism operates by running multiple self-attention heads in parallel and concatenating their outputs, $head_i$, to capture information from different subspaces. The computation formula is as follows:

$$Attention(Q, K, V) = \sigma\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{3}$$

$$MultiHead(Q, K, V) = Concat(head_1, head_2, \ldots, head_h) \cdot W^O \tag{4}$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \tag{5}$$

where σ represents the softmax function, and Q, K, and V denote the query, key, and value matrices, respectively. d_k is the dimension of the key vectors. Additionally, T denotes the transposed matrix. The variable h represents the number of heads, while W_i^Q, W_i^K, W_i^V, and W^O are learnable parameter matrices.

FFNN: The FFNN is employed to perform further transformations on the vectors at each position. It consists of two linear transformations followed by a ReLU activation function, which can be expressed mathematically as:

$$FFNN(z) = \max(0, z \cdot W_1 + b_1) \cdot W_2 + b_2 \tag{6}$$

where z represents the input of FFNN layer. W_1 and b_1 are the weight matrix and bias vector of the first Fully Connected (FC) layer, while W_2 and b_2 are the weight matrix and bias vector of the second FC layer.

3) Output Layer: The output layer of BERT model varies depending on the specific task. In the BERT pre-training phase of KDBC, the output layer is utilized to compute the loss for the Masked Language Model (MLM).

$$\mathcal{L}_{MLM} = -\sum_{t \in M} \log P\left(x_t \mid X_{\setminus M}\right) \tag{7}$$

where M is the set of masked positions, and $X_{\setminus M}$ is the input with masked tokens. x_t represents the true value of the masked word at index t.

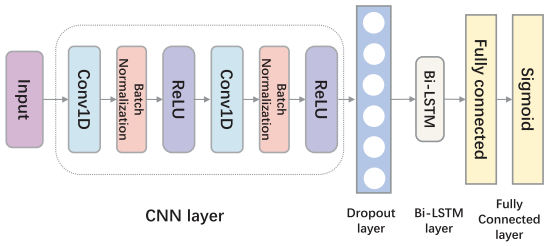


Fig. 6. The structure of CNN-BiLSTM.

Following the pre-training phase is the fine-tuning stage, where the BERT fine-tuning process adjusts the pre-trained model for specific tasks, enabling it to better adapt to and address particular challenges. The fine-tuning process primarily involves the following steps:

- Adding Task-Specific Layers: For the binary classification task aimed at IVNs outlined in this paper, the fine-tuning of KDBC model involves modifying the activation function of FC layer in the final output stage to sigmoid.
- Parameter Setting: After adding the task-specific layers, fine-tuning typically does not require training from scratch. Instead, it initializes using the parameters of the pre-trained model, which accelerates the training process and enhances model performance.
- Fine-Tuning Training: Supervised learning is conducted using labeled training data, during which the model parameters are fine-tuned. Subsequently, the loss between the true labels and the predicted labels is calculated using the cross-entropy loss function, as follows:

$$L = \frac{1}{N}\sum_{i} L_i = \frac{1}{N}\sum_{i} -\left[y_i \cdot \log\left(p_i\right) + \left(1 - y_i\right) \cdot \log\left(1 - p_i\right)\right] \quad (8)$$

where y_i represents the label for sample i, with positive class denoted as 1 and negative class as 0. p_i indicates the probability that sample i is predicted as the positive class. The model parameters are subsequently updated through back-propagation and an optimization algorithm to minimize the loss. The teacher model adjusts its parameters based on the loss to achieve better training outcomes. Finally, the logits corresponding to the optimal results are transferred to the student model.

D. Student Model

Considering the limited resource environment of automotive systems, the KDBC framework demands the student model to maintain high performance while achieving low computational complexity and a smaller model size. Therefore, for IVN traffic characterized by temporal and spatial features, the KDBC ultimately selects the CNN-BiLSTM model. By combining the strengths of CNN and BiLSTM, this model can extract local features from network traffic and capture long-term dependencies within the data, thereby enabling accurate representation of complex IVN traffic and enhancing intrusion detection accuracy [36].

As illustrated in Fig. 6, the processed IVN data is first fed into a series of meticulously designed CNN layers, which extract local features from the raw data through convolution operations. Each CNN layer is immediately followed by a Batch Normalization (BN) layer, which normalizes the output to accelerate the training process and improve model stability. Subsequently, a ReLU activation function is applied to the normalized data, introducing non-linearity that allows the model to learn more complex feature representations. After two rounds of processing with CNN, BN, and ReLU, the data is passed to the Dropout layer to mitigate the risk of overfitting.

Next, the processed data is sent to the Bi-LSTM layer. Bi-LSTM effectively captures long-term dependencies in sequence data by considering both forward and backward sequential information. This layer transforms the spatial features extracted by the convolutional layers into high-level representations over time.

Finally, the output from the Bi-LSTM layer is passed to a FC layer, which maps the features to the final output space through a weight matrix transformation. The output of the FC layer is then processed by a sigmoid activation function, compressing the values into the (0, 1) range, representing the predicted probabilities for the binary classification task. This entire process leverages the advantages of CNN in feature extraction, BN+ReLU in enhancing non-linearity and stability, and Bi-LSTM's capability in handling sequential data, collectively forming a robust model architecture.

E. Knowledge Distillation

In our work, the knowledge distillation process transfers BERT's deep semantic understanding and contextual dependencies into a lightweight CNN-BiLSTM model. This is crucial for capturing subtle temporal and structural patterns in IVN traffic. The most critical transferred knowledge includes high-efficiency learning capability in few-shot scenarios, context-aware feature representations and sequential semantic understanding, which enhance the student model's ability to detect anomalies beyond static field analysis.

In knowledge distillation, response-based knowledge distillation is one of the significant methods. On one hand, it is relatively straightforward to implement, as it focuses solely on the differences in responses between the teacher model and the student model at the output layer [37]. On the other hand, it possesses strong versatility capabilities. By imitating the predictions of the teacher model, the student model can effectively learn and adopt the teacher model's versatility capabilities. Consequently, the distillation process in KDBC employs response-based knowledge to compute the final loss.

The core idea of response-based knowledge distillation is to enable the student model to directly imitate the final prediction results of the teacher model. By minimizing the difference in responses between the student model and the teacher model at the output layer, the student model can absorb the versatility capabilities and knowledge of the teacher model.

Algorithm 2 illustrates how to utilize knowledge distillation techniques to transfer knowledge from the teacher model to the student model, thereby enhancing the performance of the student model. Initially, the inputs include the training data loader (*train_dataloader*), the teacher model

Algorithm 2 The Process of Knowledge Distillation

Input: *train_dataloader*, *teacher_model*, *student_model*
Output: *epoch_loss*

1: $epoch\_loss \leftarrow 0$
2: $\alpha \leftarrow 0.5$
3: **for** epoch in epochs **do**
4: $total\_loss \leftarrow 0$
5: **for** step, x_train, y_train in enumerate($train\_dataloader$) **do**
6: $teacher\_logits = teacher\_model(x\_train, y\_train)$
7: $student\_output$, $student\_logits$ = $student\_model(x\_train, y\_train)$
8: $p^t = sigmoid\left(\frac{teacher\_logits}{temp}\right)$
9: $q^t = sigmoid\left(\frac{student\_logits}{temp}\right)$
10: $soft\_loss = temp^2 \times KLDivLoss(p^t, q^t)$
11: $hard\_loss = BinaryCrossEntropy(student\_output, y\_train)$
12: $KDBC\_loss = \alpha \times hard\_loss + (1-\alpha) \times soft\_loss$
13: $KDBC\_loss$.backward()
14: Adam($student\_model$.parameters(), lr=2×10^{-4}, $weight\_decay = 10^{-5}$).step()
15: $total\_loss = total\_loss + KDBC\_loss$
16: **end for**
17: $epoch\_loss \leftarrow \frac{total\_loss}{len(train\_dataloader)}$
18: **end for**
19: **return:** *epoch_loss*

(*teacher_model*), and the student model (*student_model*), while the output is the average loss for each epoch (*epoch_loss*). The *epoch_loss* is initialized to 0, and the weight (α) for the distillation loss is set to 0.5 (lines 1-2).

Upon entering the training loop, each epoch is traversed, and the total loss for that epoch (*total_loss*) is initialized to 0 (lines 3-4). For each batch of training data (x_train, y_train), the current step is obtained (line 5). Predictions in the form of logits are generated using the teacher model (*teacher_logits*) (line 6), and predictions and logits are produced using the student model (*student_output* and *student_logits*) (line 7). Subsequently, the soft label probabilities (p^t and q^t) are computed using the logits from both models through the sigmoid function (lines 8-9). The soft label loss ($soft\_loss$) is calculated using the Kullback-Leibler (KL) divergence loss function (*KLDivLoss*), incorporating a temperature coefficient (*temp*) (line 10), while the hard label loss ($hard\_loss$) is computed using *BinaryCrossEntropy* function (line 11). The calculation formula for the *BinaryCrossEntropy* function can be referred to as Equation 8.

The aforementioned *KLDivLoss* function is an asymmetric measure used to quantify the difference between two probability distributions. It is commonly employed in the fields of information theory and machine learning to compare the predicted distribution with the true distribution. The definition of the *KLDivLoss* function is as follows:

$$D_{KL}(P\|Q) = \sum_o P(o) \log \frac{P(o)}{Q(o)} \tag{9}$$

where $D_{KL}(P\|Q)$ represents the KL divergence from distribution P to distribution Q. It is a non-negative value used to measure the extent to which distribution Q deviates from distribution P. $P(o)$ denotes the probability of the o th event in the target or true distribution under discrete conditions. $Q(o)$ denotes the probability of the o th event in the approximate or predicted distribution under discrete conditions.

Next, the total knowledge distillation loss ($KDBC\_loss$) is computed by combining the soft label loss and the hard label loss, adjusting the weights of the two losses using α (line 12). The total loss undergoes backpropagation (line 13), and the parameters of the student model are updated using the Adam optimizer, with the learning rate and weight decay set to prevent overfitting (line 14). The current batch loss is added to the total loss (line 15), and the total loss is normalized by the number of batches to yield the average loss for each epoch ($epoch\_loss$) (lines 16-17). Finally, the average loss for each epoch ($epoch\_loss$) is returned (lines 18-19).

V. Experimental Evaluation

In this study, the KDBC model is evaluated within specific hardware and software environments. Considering the resource limitations of actual in-vehicle environments, the knowledge distillation process of KDBC model is conducted offline on a server, after which the trained student model is loaded onto the in-vehicle gateway for real-time online testing.

This study utilized the Nvidia RTX 4090 GPU for its exceptional computational power and memory capacity, supporting the training and inference of the teacher model. The CPU, an AMD Ryzen Threadripper PRO 5995WX with 64 cores and 128 threads, ensured efficient multitasking for data loading, preprocessing, and model training. The KDBC model was developed using JupyterLab 3.4.4, which facilitates code writing, debugging, documentation, and data visualization. The implementation was done in Python 3.10.6, with libraries such as PyTorch for neural network training, NumPy for numerical computations, Pandas for data processing, and Matplotlib and Seaborn for visualization.

A. Datasets

By transferring the deep semantic knowledge from a large pre-trained BERT model (teacher) to a lightweight CNN-BiLSTM model (student), the KDBC are able to retain high detection performance while significantly reducing model complexity and training data requirements. This approach effectively leverages the representation power learned from large-scale pre-training, thereby enhancing generalization on limited-domain data. While the model still has a certain level of dependency on data, we mitigate this by employing two diverse and complementary datasets and real vehicle test for training and evaluation.

This study evaluates the datasets used for the IDS model, which include the CAN bus representative dataset, can-train-and-test [14], and the automotive Ethernet representative dataset, TOW-IDS [6]. Since the TOW-IDS dataset [6] contains normal/attack data for the gPTP protocol, and there is currently no publicly available data on in-vehicle TSN attacks, this study utilizes TOW-IDS [6] to represent both automotive Ethernet and TSN datasets.

The can-train-and-test dataset [14] was collected from real road data involving four different vehicles and six drivers, aiming to create a diverse CAN dataset for machine learning purposes. The dataset accesses CAN traffic data through the OBD-II port of each vehicle, connecting via a Korlan USB2CAN cable to convert the raw OBD-II data into USB data, thereby enabling communication through the SocketCAN subsystem and can-utils tools in Linux. For this study, sub-dataset 3 has been selected as the representative dataset for the CAN bus.

The TOW-IDS dataset [6] was released by the HCRL[1] and is specifically designed for the development and evaluation of IDSs in automotive Ethernet environments. This dataset includes both normal and abnormal data from various real driving scenarios, covering AVTP, gPTP, and UDP protocols. The abnormal dataset consists of five types of attack: frame injection, PTP synchronization, switch MAC flooding, CAN DoS, and CAN replay attacks. The data was extracted through port mirroring on a 100BASE-T1 switch and is accompanied by detailed annotations, thereby facilitating researchers in developing and validating IDSs for IVNs.

B. Evaluation Metrics

In evaluating the performance of KDBC model, this study employs a set of key metrics, as indicated by the following formulas:

$$Accuracy = \frac{TRP + TRN}{TRP + TRN + FAP + FAN} \tag{10}$$

$$Precision = \frac{TRP}{TRP + FAP} \tag{11}$$

$$Recall = \frac{TRP}{TRP + FAN} \tag{12}$$

$$F1 - Score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \tag{13}$$

where *TRP* indicates the number of normal traffic samples that are correctly classified as normal traffic. *FAN* indicates the number of normal traffic samples that are misclassified as intrusion traffic. *FAP* indicates the number of intrusion traffic samples that are misclassified as normal traffic. *TRN* indicates the number of intrusion traffic samples that are correctly classified as intrusion traffic.

Accuracy measures the proportion of correct predictions, serving as a key performance metric. The F1-Score, the harmonic mean of precision and recall, is useful for imbalanced datasets, offering a comprehensive evaluation. The confusion matrix shows the distribution of true versus predicted labels, providing an intuitive performance overview across classes.

C. Experimental Results

This study evaluates the performance of KDBC model using two datasets: the TOW-IDS [6] for automotive Ethernet and the can-train-and-test [14] for in-vehicle CAN. The experimental design aims to validate the advantages of KDBC model in enhancing the efficiency and accuracy of IDSs. It accomplishes this by comparing the performance of different models across these two datasets. Specifically, this study first compares the original BERT model, the baseline CNN-BiLSTM model, the KDBC model, and other models under two scenarios. These two scenarios include using the TOW-IDS dataset [6] alone and using a combination of the TOW-IDS [6] and can-train-and-test [14] datasets, with a training-testing split of 70%:30%. Meanwhile, the study focuses on analyzing the accuracy, F1-Score, and other key performance indicators of each model across different datasets. This section also includes a comparative analysis with the optimal solutions from existing research focused on the TOW-IDS dataset [6]. Moreover, relevant approaches were replicated using both two datasets to facilitate comparison with the KDBC model.

1) Experimental Results Under TOW-IDS Dataset: This study conducted comprehensive testing on the TOW-IDS dataset [6], with detailed results presented in Fig. 7 and 8. Specifically, Fig. 7 visually depicts the confusion matrices of three different models on this dataset, clearly illustrating the classification performance of each model. Fig. 8 further compares the key performance indicators of the three models, with the teacher model (BERT) showing exceptional results, achieving an accuracy of 0.9980 and an impressive F1-Score of 0.9971.

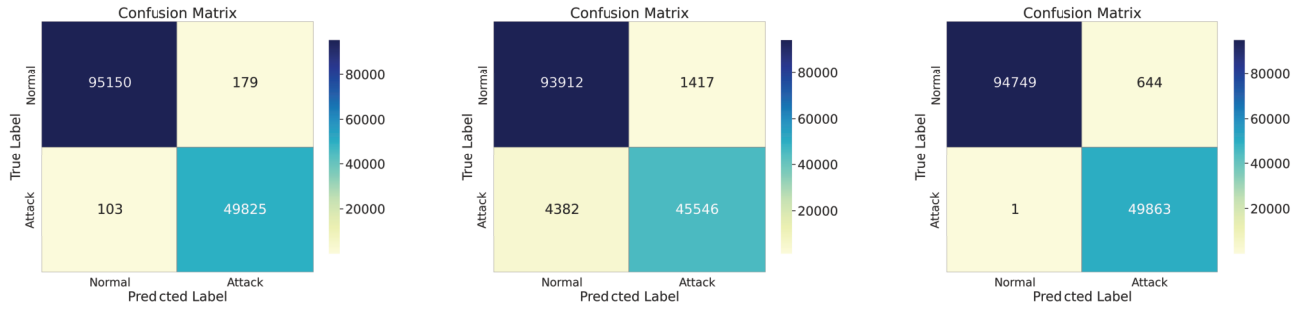
In contrast, the proposed KDBC model follows closely, demonstrating strong performance with an accuracy of 0.9966, slightly below that of the BERT model, while achieving a high F1-Score of 0.9935. Notably, the original CNN-BiLSTM model, which was not optimized through knowledge distillation, exhibited relatively weaker detection performance among the three models, with an accuracy of 0.9600 and an F1-Score of 0.9401.

The comparative analysis reveals that the KDBC model, enhanced by knowledge distillation, not only exhibits performance that is nearly indistinguishable from the teacher model BERT, but also shows significant improvement over the original CNN-BiLSTM model. Specifically, its accuracy increased by 3.66 percentage points, and its F1-Score rose by 0.0534, effectively validating the efficacy and superiority of KDBC model.

2) Experimental Results Under TOW-IDS + Can-Train-and-Test Dataset: To further validate the performance of model, this study integrated the TOW-IDS [6] and can-train-and-test [14] datasets, conducting tests on this combined dataset. The results are illustrated in Fig. 9 and 10. Fig. 9 comprehensively presents the confusion matrices for the three models on the mixed dataset, providing a clear reflection of classification accuracy. Fig. 10 systematically compares the key performance indicators of the three models, with the teacher model (BERT) maintaining its leading position, achieving an accuracy of 0.9999 and an exceptionally high F1-Score of 0.9999.

Following the BERT model is the proposed KDBC model, which is an optimized CNN-BiLSTM model enhanced through knowledge distillation. The KDBC model demonstrates outstanding performance on the mixed dataset, achieving an accuracy of 0.9996 and an F1-Score of 0.9997, showing only a slight gap from the teacher model. In contrast, the original CNN-BiLSTM model, which was not subjected to knowledge distillation, exhibited relatively weaker performance among

[1]Hacking and Countermeasure Research Lab (HCRL): https://ocslab.hksecurity.net/welcome



(a) BERT (b) CNN-BiLSTM (c) KDBC

Fig. 7. The confusion matrix of three models on TOW-IDS.

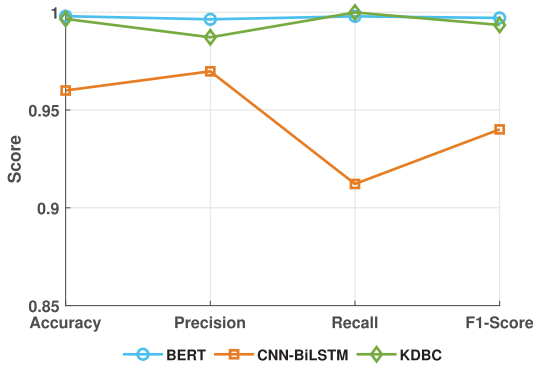


Fig. 8. The comparison of three models on TOW-IDS.

the three models, with an accuracy of 0.9658 and an F1-Score of 0.9696.

Through comparative analysis, it is evident that the KDBC model not only approaches the performance of the teacher model BERT, but also achieves a significant improvement compared to the original CNN-BiLSTM model. Specifically, the accuracy of the KDBC model increased by 3.38 percentage points, and the F1-Score also rose by 0.0301.

In comparison to the tests conducted solely on the TOW-IDS dataset [6], the results from the combined dataset show a general slight improvement across all performance metrics. This is primarily attributed to the higher discernibility of CAN bus data features compared to Ethernet data features, providing the model with richer and more learnable information. Further observation of the two result figures clearly demonstrates that the KDBC model exhibits excellent performance on both datasets, evidenced by high accuracy and F1-Scores among other key metrics. Meanwhile, the KDBC achieved 0.9700 detection accuracy with only 30% of the original training data, demonstrating that augmentation and knowledge distillation synergistically address data scarcity while ensuring efficiency. This remarkable performance is largely due to the response-based knowledge distillation strategy employed by the KDBC model. Through this strategy, the KDBC model effectively mimics the final prediction results of the BERT model, thereby learning the strong versatility capability and deep knowledge of BERT. This not only enhances the performance of the KDBC model, but also ensures stable predictive accuracy when confronted with complex datasets.

TABLE II

THE COST OF THREE MODELS ON TOW-IDS + CAN-TRAIN-AND-TEST

Methods	Parameters	Model size (MB)
BERT	109483009	417.64
CNN-BiLSTM	614913	2.35
KDBC	**264833**	**1.01**

TABLE III

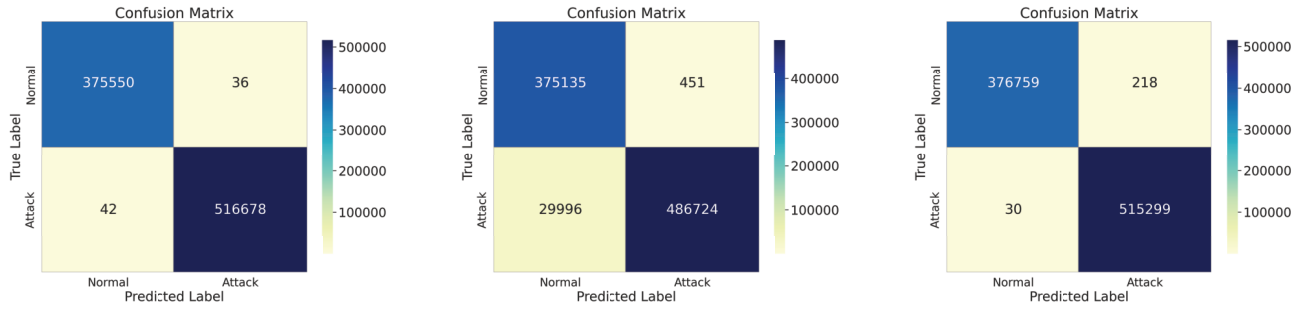
THE COMPARISON OF KDBC AND OTHER LITERATURE UNDER TOW-IDS DATASET

Methods	Accuracy	F1-Score	Model size (MB)
TOW-IDS (with Wavelet transform) [6]	0.9965	**0.9974**	11.3
ResNet50 [6]	0.9651	0.9785	270.3
EfficientNetB0 [6]	0.9603	0.9776	46.8
Multi-stage IDS [19]	0.9962	0.9960	-
Semi-supervised model [38]	0.9700	0.9500	-
KDBC	**0.9966**	0.9935	**1.01**

3) The Cost of KDBC: Given the limited computational resources available in vehicles, there is a stringent requirement for lightweight detection models. Therefore, this study not only focuses on the performance of the models, but also analyzes the sizes and parameter counts of the three models. It is evident from Table II that while the BERT teacher model excels across all performance metrics, it is significantly large, with a size of 417.64 MB and a staggering parameter count of 100 million. In contrast, both the CNN-BiLSTM and the KDBC models are notably more streamlined, with sizes of only 2.35 MB and 1.01 MB, respectively, and parameter counts limited to 614,913 and 264,833.

In the context of pursuing lightweight models, the KDBC model demonstrates significant advantages. It not only maintains high levels of detection accuracy and F1-Score among key metrics, but also successfully minimizes computational overhead through structural optimization.

4) The Comparison With Existing Research Methods: To demonstrate the advancements of KDBC model, this study compares it with models proposed in other literature. The performance metrics (accuracy, F1-score and model size) of the KDBC model are obtained by training and testing our model directly on the original, unmodified TOW-IDS and can-train-and-test datasets. For the baseline schemes listed



(a) BERT (b) CNN-BiLSTM (c) KDBC

Fig. 9. The confusion matrix of three models on TOW-IDS + can-train-and-test.

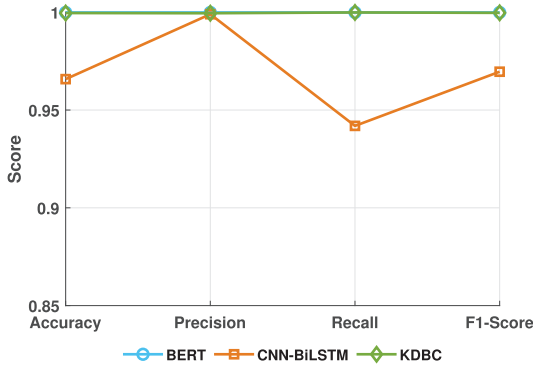


Fig. 10. The comparison of three models on TOW-IDS + can-train-and-test.

TABLE IV
THE COMPARISON OF KDBC AND OTHER LITERATURE UNDER CAN-TRAIN-AND-TEST DATASET

Methods	Accuracy	F1-Score	Model size (KB)
Gaussian Naive Bayes [14]	0.8839	0.9361	0.16
Logistic Regression [14]	0.9565	0.9539	**0.044**
Support Vector Machine [14]	0.9976	0.0018	0.86
Multi-Layer Perceptron [14]	0.9666	0.9594	4.7
KDBC	**0.9986**	**0.9978**	17.7

in Tables III and IV, the reported results are primarily extracted from their respective original papers, in which the same datasets are used under comparable conditions. We carefully select the results that used the original dataset splits without alterations to ensure fairness and consistency. Regarding the selection of comparison methods, we focused on well-established and widely cited schemes in the field of IVN intrusion detection. These include models that either (1) utilized the TOW-IDS [6] or can-train-and-test datasets in their original form, or (2) adopted deep learning-based architectures, such as CNN-LSTM and BERT variants, which are directly comparable to our proposed approach.

For the TOW-IDS dataset [6], the comparison includes TOW-IDS (with Wavelet Transform) [6], ResNet50 [6], EfficientNetB0 [6], Multi-stage IDS [19], and a semi-supervised model [38], alongside the KDBC model. As shwn in Table III, it is evident that the KDBC model achieves the highest accuracy, with its F1-Score also remaining at a high level, slightly lower than that of TOW-IDS (with Wavelet Transform) [6] and Multi-stage IDS [19]. However, the size of KDBC model is only one-tenth that of TOW-IDS (with Wavelet Transform) [6], clearly showcasing its advantages in terms of lightweight design.

TABLE V
THE COMPARISON OF KDBC AND OTHER LITERATURE UNDER TOW-IDS + CAN-TRAIN-AND-TEST DATASET

Methods	Accuracy	F1-Score	Model size (MB)
ECF-IDS [15]	0.9997	0.9999	108.3
CC-IDPS [39]	0.9998	0.9999	108.1
CNN-LSTM [11]	0.9458	0.9123	1.86
CNN-LSTM (with attention) [11]	0.9679	0.9542	2.42
KDBC	0.9996	0.9997	**1.01**

For the can-train-and-test dataset [14], we compared several baseline methods, including Gaussian Naive Bayes (GNB) [14], Logistic Regression (LR) [14], Support Vector Machine (SVM) [14], and Multi-Layer Perceptron (MLP) [14]. As shown in the Table IV, although LR has the smallest model size at only 0.044 KB, its detection performance is relatively poor. The SVM achieved a high accuracy of 0.9976, but its F1-score was extremely low at just 0.0018. The best overall performance was achieved by the proposed KDBC model, which significantly outperformed the other methods in both detection accuracy and F1-score, while maintaining a reasonable model size of 17.7 KB.

For the hybrid dataset of TOW-IDS [6] and can-train-and-test [14], this study reproduced the ECF-IDS [15], CC-IDPS [39], CNN-LSTM [11] and CNN-LSTM (with attention) [11] models to compare with the KDBC model. The comparison results are presented in Table V. It is clear that CC-IDPS [39] achieved the best performance, with an accuracy of 0.9998 and an F1-Score of 0.9999. Meanwhile, the ECF-IDS [15], CC-IDPS [39], and KDBC models all maintained high detection accuracy and F1-scores, outperforming both the CNN-LSTM [11] and CNN-LSTM (with attention) [11] models. Notably, the model size of KDBC is less than 1% of that of ECF-IDS [15] and CC-IDPS [39], making it significantly more lightweight while still delivering superior performance.

D. Real Scenarios Test

The practical applicability of the KDBC model was evaluated in two phases. In the initial stage, the model was

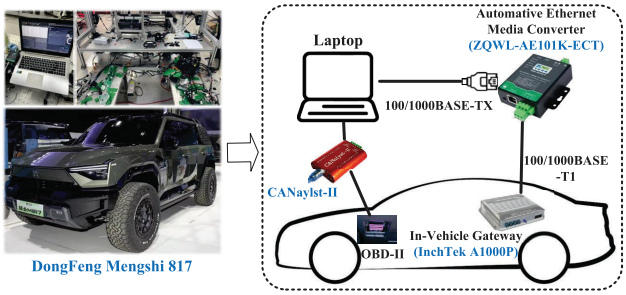


Fig. 11. Test of real scenarios and data.

trained and tested in a laboratory environment by connecting it to an InchTek A1000Plus in-vehicle security gateway.[2] The A1000Plus is an in-vehicle security gateway product that supports automotive-grade Ethernet and CAN hybrid networks. It integrates a high-performance automotive-grade gateway chip and a security system foundation chip. Its software is based on the AUTOSAR architecture, offering high reliability and meeting functional safety requirements. Once the model achieved stable performance, we proceeded to the second stage, where it was deployed and validated in a real vehicle environment[3] (DongFeng Mengshi 817). Normal traffic was collected from the real vehicle under various operating conditions, such as ignition, driving, braking, and idling.

As shown in Fig. 11, the deployed system monitored multiple in-vehicle communication channels, including Ethernet, TSN, and CAN bus data, to verify the effectiveness of the model in real-world conditions. At the same time, we injected relevant attack data and collected data from the in-vehicle CAN bus and automotive Ethernet using a laptop.

For the CAN bus tests, we used a laptop connected to the vehicle's OBD-II interface via CANalyst-II. We then injected DoS, spoof, replay, and fuzzy attack data into the in-vehicle CAN bus. The specific details of each attack were as follows:

- **DoS attack:** Inject 60274 blank messages with both CAN ID and DATA set to 0 every 0.1 milliseconds.
- **Gear/RPM Spoof attack:** Intercept CAN messages containing engine RPM or gear information, modify the engine RPM or gear data fields in the message, and resend it to the CAN bus.
- **Replay attack:** Replay the data packets captured in the previous 10 seconds every 20 seconds.
- **Fuzzy attack:** Inject messages of totally random CAN ID and DATA values every 0.1 milliseconds.

For the automotive Ethernet tests, we used a laptop connected to the in-vehicle gateway through an automotive Ethernet media converter. Using tools like hping3, Macof, LinuxPTP, and nmap on Kali Linux, we injected DoS, frame injection, switch MAC flooding, PTP synchronization, sniffing, and scanning attacks into the automotive Ethernet.

Specifically, active sniffing attacks are conducted using Ettercap and Scapy, involving techniques such as ARP spoofing, MAC flooding, and DNS poisoning to intercept and manipulate network traffic, thereby disrupting normal communication patterns. Scanning attacks are carried out with Nmap, Wireshark, and Burp Suite, including port scanning, protocol/service fingerprinting, CAN ID probing, and TSN QoS analysis to identify active nodes, services, and time-sensitive communication paths within the IVN.

[2]The in-vehicle gateway and related technologies used in this experiment were provided by Beijing Inchtek Technology Co., Ltd.

[3]The real test vehicle and related technologies used in this experiment were provided by DongFeng Motor Group Co., Ltd.

TABLE VI
THE COMPOSITION OF IN-VEHICLE GATEWAY TEST DATASET

Type	Amount
Normal	405071
Ethernet DoS	60274
Ethernet Frame Injection	41057
Ethernet Switch MAC flooding	31319
TSN PTP Synchronization	26022
CAN DoS	31000
CAN RPM Spoof	17800
CAN Gear Spoof	19500
CAN Replay	13600
CAN Fuzzy	12600
Sniffing	17577
Scanning	18978

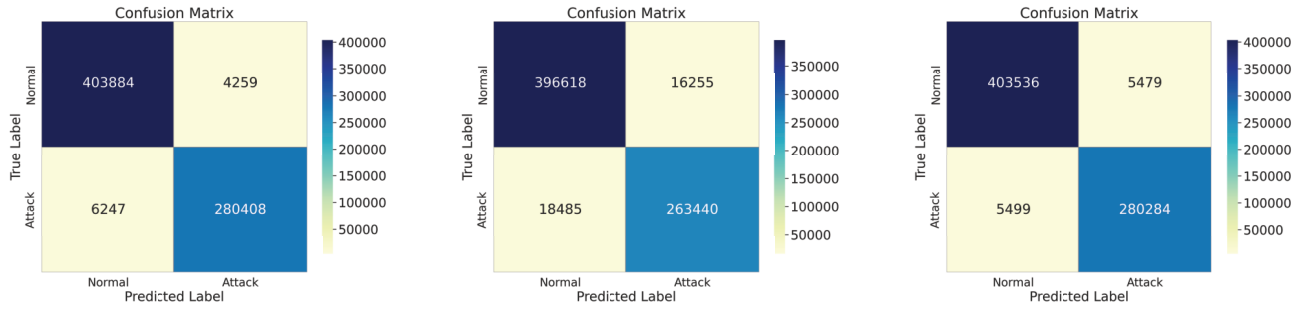
TABLE VII
THE COMPARISON OF KDBC AND OTHER LITERATURE UNDER REAL SCENARIOS

Methods	Accuracy	F1-Score	Model size (MB)
ECF-IDS [15]	0.9849(↓ 0.0148)	0.9825(↓ 0.0174)	108.3
CC-IDPS [39]	0.9852(↓ 0.0146)	0.9827(↓ 0.0172)	108.1
CNN-LSTM [11]	0.9018(↓ 0.0440)	0.8732(↓ 0.0391)	1.86
CNN-LSTM (with attention) [11]	0.9283(↓ 0.0396)	0.9176(↓ 0.0366)	2.42
KDBC	0.9842(↓ 0.0154)	0.9816(↓ 0.0181)	**1.01**

Unlike the TOW-IDS [6] and can-train-test [14] datasets, the actual test data (as shown in Table VI) included novel network attacks that were not present in the training dataset (e.g., sniffing and scanning attacks). As a result, the testing difficulty increased due to the need to identify these new attack types. Therefore, as shown in Fig. 12, the performance metrics of the KDBC model in real scenarios are lower compared to those in the TOW-IDS dataset [6]. Nevertheless, the KDBC model still maintains a high level of detection performance, achieving an accuracy of 0.9842, precision of 0.9715, recall of 0.9874, and an F1 score of 0.9816.

The comparative test results of the ECF-IDS [15], CC-IDPS [39], CNN-LSTM [11], CNN-LSTM (with attention) [11] and KDBC models in real scenarios are shown in Table VII. Compared to the results in Table V, the performance of all five models declined in real testing scenarios, with the CNN-LSTM [11] model showing the most significant drop in accuracy by 0.0440. The accuracy of both the ECF-IDS [15], CC-IDPS [39] and KDBC models decreased by 0.0148, 0.0146 and 0.0154 respectively. Although the performance of the KDBC model slightly lags behind the ECF-IDS [15] model, the size of the KDBC model is significantly smaller than that of the ECF-IDS [15].

During the real scenarios testing process, the KDBC model displayed a certain level of versatility in detecting anomalies associated with novel attacks, which can be attributed to the high-quality text representations provided by the BERT



(a) BERT (b) CNN-BiLSTM (c) KDBC

Fig. 12. The confusion matrix of three models on real vehicle test.

teacher model. These representations helped the student model (CNN-BiLSTM) enhance its understanding and processing of input data, enabling it to learn more comprehensive features and patterns for accurate anomaly detection.

VI. Conclusion

This paper addresses security challenges in IVNs within intelligent transportation systems, particularly regarding new technologies and complex network scenarios. It introduces a lightweight and efficient anomaly traffic detection method named KDBC. The KDBC leverages knowledge distillation to transfer the deep semantic knowledge of BERT into a more efficient CNN-BiLSTM model, reducing computational and storage costs without compromising performance. Experimental results show KDBC's strong versatility and superior performance in detecting anomalies across Ethernet and CAN network data, validated in a real in-vehicle gateway environment, offering a practical solution for IVNs security. Future work will refine KDBC by exploring advanced distillation techniques to enhance its real-time detection accuracy and efficiency.

References

[1] H. Zhang, K. Zeng, and S. Lin, "Federated graph neural network for fast anomaly detection in controller area networks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1566–1579, 2023.

[2] E. Levy, A. Shabtai, B. Groza, P.-S. Murvay, and Y. Elovici, "CAN-LOC: Spoofing detection and physical intrusion localization on an in-vehicle CAN bus based on deep features of voltage signals," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 4800–4814, 2023.

[3] W. Wu et al., "A survey of intrusion detection for in-vehicle networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 919–933, Mar. 2020.

[4] Z. Niu, J. Xue, Y. Wang, T. Lei, W. Han, and X. Gao, "QARF: A novel malicious traffic detection approach via online active learning for evolving traffic streams," *Chin. J. Electron.*, vol. 33, no. 3, pp. 645–656, May 2024.

[5] Z. Deng, J. Liu, Y. Xun, and J. Qin, "IdentifierIDS: A practical voltage-based intrusion detection system for real in-vehicle networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 661–676, 2024.

[6] M. L. Han, B. I. Kwak, and H. K. Kim, "TOW-IDS: Intrusion detection system based on three overlapped wavelets for automotive Ethernet," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 411–422, 2023.

[7] M. H. Khan, A. R. Javed, Z. Iqbal, M. Asim, and A. I. Awad, "DivaCAN: Detecting in-vehicle intrusion attacks on a controller area network using ensemble learning," *Comput. Secur.*, vol. 139, Apr. 2024, Art. no. 103712.

[8] M. D. Hossain, H. Inoue, H. Ochiai, D. Fall, and Y. Kadobayashi, "An effective in-vehicle CAN bus intrusion detection system using CNN deep learning approach," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2020, pp. 1–6.

[9] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, Jan. 2020, Art. no. 100198.

[10] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in *Proc. 16th Annu. Conf. Privacy, Secur. Trust (PST)*, Aug. 2018, pp. 1–6.

[11] H. Sun, M. Chen, J. Weng, Z. Liu, and G. Geng, "Anomaly detection for in-vehicle network using CNN-LSTM with attention mechanism," *IEEE Trans. Veh. Technol.*, vol. 70, no. 10, pp. 10880–10893, Oct. 2021.

[12] W. Lo, H. Alqahtani, K. Thakur, A. Almadhor, S. Chander, and G. Kumar, "A hybrid deep learning based intrusion detection system using spatial–temporal representation of in-vehicle network traffic," in *Proc. Veh. Commun. (VEH COMMUN)*, vol. 35, Mar. 2022, p. 100076.

[13] X. Zhang, L. Hao, G. Gui, Y. Wang, B. Adebisi, and H. Sari, "An automatic and efficient malware traffic classification method for secure Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8448–8458, Mar. 2024.

[14] B. Lampe and W. Meng, "Can-train-and-test: A curated CAN dataset for automotive intrusion detection," *Comput. Secur.*, vol. 140, May 2024, Art. no. 103777.

[15] S. Li, Y. Cao, H. J. Hadi, F. Hao, F. B. Hussain, and L. Chen, "ECF-IDS: An enhanced cuckoo filter-based intrusion detection system for in-vehicle network," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 4, pp. 3846–3860, Aug. 2024.

[16] E. Nwafor and H. Olufowobi, "CANBERT: A language-based intrusion detection model for in-vehicle networks," in *Proc. 21st IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2022, pp. 294–299.

[17] L. Du, Z. Gu, Y. Wang, and C. Gao, "Open world intrusion detection: An open set recognition method for CAN bus in intelligent connected vehicles," *IEEE Netw.*, vol. 38, no. 3, pp. 76–82, May 2024.

[18] R. Islam, M. K. Devnath, M. D. Samad, and S. M. J. Al Kadry, "GGNB: Graph-based Gaussian naive Bayes intrusion detection system for CAN bus," *Veh. Commun.*, vol. 33, Jan. 2022, Art. no. 100442.

[19] L. F. M. da Luz, P. F. D. Araujo-Filho, and D. R. Campelo, "Multi-stage deep learning-based intrusion detection system for automotive Ethernet networks," *Ad Hoc Netw.*, vol. 162, Sep. 2024, Art. no. 103548.

[20] S. Jeong, H. K. Kim, M. L. Han, and B. I. Kwak, "AERO: Automotive Ethernet real-time observer for anomaly detection in in-vehicle networks," *IEEE Trans. Ind. Informat.*, vol. 20, no. 3, pp. 4651–4662, Mar. 2024.

[21] D. Ergenç, R. Schenderlein, and M. Fischer, "TSNZeek: An open-source intrusion detection system for IEEE 802.1 time-sensitive networking," in *Proc. IFIP Netw. Conf. (IFIP Netw.)*, Jun. 2023, pp. 1–6.

[22] P. Meyer, T. Häckel, F. Korf, and T. C. Schmidt, "Network anomaly detection in cars based on time-sensitive ingress control," in *Proc. IEEE 92nd Veh. Technol. Conf. (VTC-Fall)*, Nov. 2020, pp. 1–5.

[23] Z. Chen et al., "Digital forensics for automotive intelligent networked terminal devices," *IEEE Trans. Veh. Technol.*, vol. 73, no. 4, pp. 5128–5138, Apr. 2024.

[24] M. De Vincenzi et al., "A systematic review on security attacks and countermeasures in automotive Ethernet," *ACM Comput. Surveys*, vol. 56, no. 6, pp. 1–38, Jun. 2024.

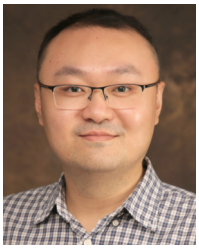
[25] H. Kim, W. Yoo, S. Ha, and J.-M. Chung, "In-vehicle network average response time analysis for CAN-FD and automotive Ethernet," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 6916–6932, Jun. 2023.

[26] Z. King, "Investigating and securing communications in the controller area network (CAN)," in *Proc. Int. Conf. Comput., Netw. Commun. (ICNC)*, Jan. 2017, pp. 814–818.
[27] S. Gao, L. Zhang, L. He, X. Deng, H. Yin, and H. Zhang, "Attack detection for intelligent vehicles via CAN-bus: A lightweight image network approach," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 16624–16636, Dec. 2023.
[28] M. H. Shahriar, Y. Xiao, P. M. Salazar, W. Lou, and T. Y. Hou, "CANShield: Deep learning-based intrusion detection framework for controller area networks at the signal-level," *IEEE Internet Things J. (IoTJ)*, vol. 9, no. 6, pp. 4528–4539, Jan. 2022.
[29] A. Morato, E. Ferrari, S. Vitturi, F. Tramarin, C. Zunino, and M. Cheminod, "A TSN-based approach to combine real-time CAN network with in-vehicle Ethernet," in *Proc. IEEE Int. Workshop Metrol. Automot. (MetroAutomotive)*, Jun. 2024, pp. 12–17.
[30] J. Zhou, S. Li, Y. Cao, H. J. Hadi, and H. Lin, "Robust intrusion detection system in CAN bus through multi-scale feature fusion," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2024, pp. 1316–1321.
[31] W. Su et al., "Towards All-in-one pre-training via maximizing multi-modal mutual information," *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 2023, pp. 15888–15899, Jan. 2022.
[32] M. A. Islam, M. Kowal, S. Jia, K. G. Derpanis, and N. D. B. Bruce, "Position, padding and predictions: A deeper look at position information in CNNs," *Int. J. Comput. Vis.*, vol. 132, no. 9, pp. 1–22, Sep. 2024.
[33] N. Fei, Y. Gao, Z. Lu, and T. Xiang, "Z-score normalization, hubness, and few-shot learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 142–151.
[34] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. naacL-HLT*, Jan. 2018, p. 2.
[35] Z. Zhang et al., "Semantics-aware BERT for language understanding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 5, 2020, pp. 9628–9635.
[36] J. Sinha and M. Manollas, "Efficient deep CNN-BiLSTM model for network intrusion detection," in *Proc. 3rd Int. Conf. Artif. Intell. Pattern Recognit. (AIPR)*, Jun. 2020, pp. 223–231.
[37] J. H. Cho and B. Hariharan, "On the efficacy of knowledge distillation," in *Proc. ICCV*, Oct. 2019, pp. 4794–4802.
[38] K. H. Shibly, M. D. Hossain, H. Inoue, Y. Taenaka, and Y. Kadobayashi, "A feature-aware semi-supervised learning approach for automotive Ethernet," in *Proc. IEEE Int. Conf. Cyber Secur. Resilience (CSR)*, Jul. 2023, pp. 426–431.
[39] S. Li, Y. Cao, Y. Zhang, T. Liao, F. Yan, and H. Lin, "A cloud collaborative-based intrusion detection and prevention system for IVN," *IEEE Trans. Cognit. Commun. Netw.*, vol. 6, no. 4, p. 1, Nov. 2025.

Sifan Li received the B.S. degree from Nanchang University in 2021 and the M.S. degree from Wuhan University in 2024. He is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Wuhan University. His research interests include in-vehicle network security, the IoT intrusion detection, and aerospace-sky-earth cybersecurity.

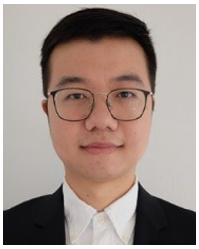
Yue Cao (Senior Member, IEEE) received the Ph.D. degree from the Institute for Communication Systems (ICS) formerly known as the Centre for Communication Systems Research, University of Surrey, Guildford, U.K., in 2013. Further to his Ph.D. study, he had conducted a Research Fellow with the University of Surrey and academic faculty with Northumbria University, U.K., Lancaster University, U.K., and Beihang University, Beijing, China. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University, Wuhan, China. His research interests include intelligent transport systems, including e-mobility, V2X, and edge computing.

Guojun Peng received the Ph.D. degree from Wuhan University, Wuhan, China, in 2008. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. His research interests inlcude system security and trust computing. He is a member of CCF and CSAC.



Meng Li (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from the School of Computer Science and Technology, Beijing Institute of Technology (BIT), China, in 2019. He is currently an Associate Professor and the Dean Assistant of the School of Computer Science and Information Engineering, Hefei University of Technology (HFUT), China. He is also a Post-Doctoral Researcher with the Department of Mathematics and the HIT Center, University of Padua, Italy, where he is with the Security and PRIvacy Through Zeal (SPRITZ) Research Group led by Prof. Mauro Conti (IEEE Fellow). He was sponsored by the ERCIM "Alain Bensoussan" Fellowship Program to conduct post-doctoral research supervised by Prof. Fabio Martinelli at CNR, Italy, from October 2020 to March 2021. He was sponsored by China Scholarship Council (CSC) for the joint Ph.D. study supervised by Prof. Xiaodong Lin (IEEE Fellow) with the Broadband Communications Research (BBCR) Laboratory, University of Waterloo, and Wilfrid Laurier University, Canada, from November 2017 to August 2018. His research interests include security, privacy, applied cryptography, blockchain, TEE, and the Internet of Vehicles. In this area, he has published 80 papers in international peer-reviewed journals and conferences, including IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, TODS, IEEE TRANSACTIONS ON SERVICES COMPUTING, COMST, ISSTA, MobiCom, ACISP, ICICS, SecureComm, TrustCom, ICC, and IPCCC. He is a Senior Member of CIE, CIC, and CCF. He is an Associate Editor of IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, and IEEE INTERNET OF THINGS JOURNAL.

Wei Sun received the degree from the Department of Systems Engineering and Applied Mathematics, Chongqing University. He is currently working at the Research and Development Institute, Dongfeng Motor Group Corporation Ltd., as the Chief Digital Engineer. For many years, he has been in charge of informatization and digital planning in the research and development field, the implementation of research and development information system projects, and connected vehicle cybersecurity work. He has rich experience in informatization and digitalization in the research and development field as well as in the cybersecurity of connected vehicles.



Luan Chen (Member, IEEE) received the Ph.D. degree from the Conservatoire National des Arts et Métiers (CNAM-Paris) in 2020. He is currently an Associate Professor with the Information, Communications, and Imaging (ICI) Group, ETIS Laboratory (UMR 8051), a joint research laboratory of CY Cergy Paris University, ENSEA, and CNRS, France. His research interests include wireless indoor localization/sensing, statistical signal processing (RF/biomedical), mobile computing for IoT, and machine learning technologies.